BÖLÜM 2: PLANLAR VE SINIRLAR — KARMAŞIKLIĞI PARÇALARA AYIRMAK 9 dk • DERS 8

# Parça 8: Kutsal Bir Sözleşme Olarak API Tasarımı: REST, gRPC ve GraphQL Karşılaştırması

*İstemci uygulamaların Cuma akşamı patlamadan güvenle tüketebileceği arayüzler tasarlamak.*

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> 2 milyon aktif mobil kullanıcısı olan bir mobil bankacılık uygulamasında backend geliştirici Cuma akşamı `/api/v1/hesaplar` ucundaki `hesap_no` alanını temiz kod aşkına `hesapNo` olarak güncelledi. Web ekibi hemen güncellemeyi alıp canlıya çıktı. Ancak App Store ve Google Play'den uygulamasını henüz güncellememiş 1.4 milyon kullanıcının hesap bakiyeleri Cumartesi sabahı birden 0.00 TL görünmeye başladı! Bankanın çağrı merkezi kilitlendi, hisseler değer kaybetti. Kullanıcıların telefonundaki eski uygulamayı zorla güncelleyemeyeceğiniz için sistem acil gece yarısı yamasıyla geri alınmak zorunda kaldı.

## Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Dahili bir fonksiyon imzası size aittir; ancak yayınlanmış bir API tüketicilerinize aittir. Bir API mobil cihazlara, üçüncü parti ortaklara veya genel kullanıma açıldığı anda geri alınamaz bir sözleşmeye dönüşür. Kullanıcıların telefonlarını ne zaman güncelleyeceğini kontrol edemezsiniz. Usta bir mimar API'leri geriye dönük mutlak uyumluluk (backward compatibility) ve toleranslı okuma ilkeleriyle tasarlar.

## Temel Mimari İlkeler ve Zihinsel Modeller

### Protokol Karşılaştırması

- **REST (HTTP/JSON):** Evrensel, tarayıcıda kolay debug edilir, genel geliştirici ekosistemleri ve standart CRUD işleri için ideal.
- **gRPC (HTTP/2 üzerinde Protobuf):** İkili serileştirme ile ultra yüksek hız, sıkı şema sözleşmeleri, iki yönlü akış. Mikroservislerin iç haberleşmesinin tartışmasız kralı.
- **GraphQL:** İstemcinin ihtiyacı olan veriyi kendisinin seçtiği yapı. Mobil uygulamalarda gereksiz veri indirmeyi (over-fetching) engeller, zengin dashboard'lar için harikadır.

### API Evriminin Altın Kuralları

1. **Yalnızca Ekleme Yapılabilir (Additive Changes):** Yeni alanlar ekleyebilirsiniz; fakat mevcut bir alanı silemez veya yeniden adlandıramazsınız.
2. **Toleranslı Okuyucu Kalıbı:** İstemciler JSON yanıtında tanımadıkları yeni alanları yoksaymalıdır.
3. **Idempotency Anahtarları:** Veri değiştiren tüm uçlar (`POST /odemeler`) bir `Idempotency-Key` başlığı kabul ederek mükerrer çekimleri engellemelidir.

## Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Veritabanını Dışarı Sızdırmak:** `SELECT * FROM kullanicilar` sonucunu doğrudan JSON olarak dönüp kullanıcının şifre özetlerini ve iç tablo kolonlarını internete açmak.
- **GraphQL Vahşi Batısı:** Derinlik sınırı koymadan genel internete açılan GraphQL uçları yüzünden kötü niyetli kişilerin iç içe sorgularla sunucuları kilitlemesi.

- **Sessiz Kırıcı Değişiklik:** Versiyon numarasını değiştirmeden bir alanın biçimini (örn: ISO 8601 tarih yerine milisaniye dönmek) değiştirmek.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Kod yazmadan önce OpenAPI (Swagger) veya Protobuf sözleşmesini yazıp onaylatın (API-First Tasarım).
> - **2. Adım:** CI/CD sürecine otomatik sözleşme testleri (Pact gibi) entegre ederek kırıcı değişikliklerin canlıya gitmesini engelleyin.
> - **3. Adım:** Tüm kritik ödeme ve sipariş uçlarında `Idempotency-Key` başlığını zorunlu hale getirin.

## Mimari Çıkarımlar ve Hile Tablosu

- API'ler sonsuza dek yaşar: Biri o uca göre mobil uygulama derlediğinde, o şemayı yıllarca desteklemek zorunda kalırsınız.
- Dahili servis haberleşmesinde gRPC'nin hızını; dış dünyada REST veya GraphQL'in esnekliğini kullanın.
- Veritabanı kolonundaki bir isim değişikliğinin dış dünyadaki API kontratını kırmasına asla izin vermeyin.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Şirketinizdeki ana API uçlarından birini inceleyin. Versiyonlama nasıl yapılıyor? 3 yıl önce yazılmış eski bir mobil istemci bugün o uca istek atsa çalışmaya devam eder mi?